A COMPREHENSIVE STUDY REPORT ON

Fill-Mask and Masked Language Modelling: Foundations, Methods and Current Developments

Natural Language Processing

Assignment 1

Topic 4: Fill-Mask

Chapter 2: Application Topics

Literature reviewed: October 13, 2026

Abstract

Fill-mask prediction uses the visible context of a sentence to rank tokens that could occupy a hidden position. This report explains the task through language-model objectives, BERT's bidirectional representations, tokenization, vocabulary scores and the distinction between training and inference. A single-mask BERT assistant provides a concrete implementation with explicit validation, ranked candidate records and browser and command-line interfaces. The retained problem statement, design, proposed methodology, implementation and result sections connect the software workflow to its theoretical basis. Real inference on twelve hand-authored sentences recovered the original token in six top-1 predictions and ten top-5 lists. These measurements describe a small demonstration rather than general language competence. The document also distinguishes exact recovery, grammatical acceptability and factual correctness. A current-status review, based on representative literature checked on October 13, 2026, discusses modern encoders, masked diffusion and research combining masked and autoregressive infilling. The report identifies multi-token completion, ambiguity, domain transfer and reproducible evaluation as continuing issues and provides a foundation for understanding the wider field beyond the selected BERT checkpoint.

Keywords: fill-mask; BERT; masked language modelling; contextual prediction; infilling

Index

Abstract 2

1. Introduction 4

2. Problem Statement 7

3. Design 9

4. Proposed Methodology 12

5. Implementation 14

6. Result 16

7. Current Status and Future Directions 18

8. Reference 20

1. Introduction

Predicting missing text is a basic problem in language processing. A reader completing "She drank a glass of __" uses grammar, meaning and experience to consider plausible replacements. A computational system performs a related task by assigning scores to possible tokens given the surrounding text. Fill-mask is the application of a masked language model to a sentence containing an explicit marker at the missing position.

The task offers a useful introduction to contextual language representations because its input and output are easy to inspect. Nevertheless, a fluent replacement does not necessarily identify the word originally removed or verify a factual claim. Understanding fill-mask requires attention to tokenization, the training objective, probability distributions and the difference between statistical plausibility and correctness.

1.1 Language modelling and contextual prediction

A language model represents patterns in sequences of text. Different training objectives define what information is available when predicting a token. An autoregressive model predicts the next token from preceding context. A masked language model predicts hidden tokens using the visible surrounding sequence. An encoder-decoder model can learn to reconstruct missing spans or transform an input into another sequence.

Table 1. Distinguishing language-model formulations relevant to completion.

Formulation	Prediction target	Available context in the basic formulation
Autoregressive language modelling	Next token	Previously available tokens
Masked language modelling	Hidden token positions	Visible tokens around the masks
Sequence-to-sequence reconstruction	Missing span or output sequence	Encoded input and previously produced output

These objectives should not be confused with user-interface features. A model trained for next-token prediction can be prompted with text describing a gap, but that is different from a standard bidirectional masked-token computation. Systems designed for infilling may reorganize inputs or combine additional mechanisms. The present implementation specifically uses BERT's masked-language-model head, not a chat-generation interface.

1.2 BERT and bidirectional representations

BERT stands for Bidirectional Encoder Representations from Transformers. Its Transformer encoder combines contextual information from both sides of a token position. The original model was pretrained using masked language modelling and a next-sentence prediction objective [1]. Our application uses the masked-token capability of an existing checkpoint; it does not repeat pretraining.

Bidirectional context is especially useful when information after a gap changes the likely answer. In "She put the [MASK] on her head," the phrase after the mask narrows the possibilities. This sentence is an explanatory example, not a measured prediction. A model must still distinguish several plausible replacements and may fail when the sentence depends on unusual knowledge or an unfamiliar domain.

1.3 Tokens, vocabulary and the mask marker

The selected BERT tokenizer uses vocabulary tokens that may represent full words, word fragments or punctuation. It converts the input into numeric identifiers and adds required special tokens. A token identifier names a vocabulary entry; it is not its score or rank. The readable token and the decoded sentence are separate representations of the prediction.

The marker [MASK] indicates the position to predict. One mask represents one vocabulary token. A missing phrase may require several tokens, and some individual words also split into several pieces. Therefore, replacing one mask cannot be assumed to reconstruct an arbitrary word or phrase. Our program explicitly restricts the task to one mask and validates that the encoded input contains one mask position.

1.4 Masked language modelling and inference

During masked-language-model training, selected token information is hidden or corrupted and the model learns to recover the original targets. The targets can be obtained from the original text itself, making this a self-supervised objective. BERT's original training recipe selected 15% of token positions and used a mixture of masking, random replacement and unchanged selected tokens [1]. The application's single explicit mask is a prediction-time request, not a reproduction of every detail of that training procedure.

Training adjusts weights to reduce prediction loss. Inference uses the learned weights to score candidates for a new request. The twelve sentences stored in our project are demonstration evaluation inputs; they are not used to update BERT. Calling predict repeatedly does not teach the model new information, even though the application reuses the loaded model object.

1.5 Model selection and applications

The checkpoint google-bert/bert-base-uncased is selected because it directly supports masked-language-model inference [2]. Uncased processing normalizes capitalization, so decoded output may appear in lowercase. The project targets short English sentences and should not be treated as a validated multilingual completion tool.

Potential educational uses include vocabulary suggestions, contextual gap exercises and experiments with ambiguity. Fill-mask can also support research into language representations and domain adaptation. These are possible applications of the technique, not proof that the current demonstration is suitable for grading learners or making factual decisions. A responsible interface presents alternatives and lets the reader inspect them in context.

2. Problem Statement

The problem is to suggest replacements for exactly one masked token in an English sentence using its surrounding context. The system must validate the sentence and requested suggestion count, obtain a ranked candidate list and show how each candidate changes the sentence. The educational objective is to understand contextual prediction and the interpretation of masked-position scores.

2.1 Objectives

The objectives are to explain the masked-language-model formulation, design a single-mask application, implement reliable input checks, return top-k vocabulary candidates and record results with their model configuration. Evaluation must distinguish exact recovery of an original target from the acceptability of alternative completions. A further objective is to place the BERT example within the broader development of encoders and text-infilling methods.

2.2 Scope and constraints

The implemented version accepts a non-empty sentence containing exactly one literal [MASK]. The requested top-k must be an integer from 1 to 10. The encoded input must contain one mask token and fit the tokenizer's sequence limit. CPU execution is the default, and an optional checkpoint revision can identify an exact model version.

The system does not generate unrestricted phrases, choose how many tokens a gap needs or jointly model several replacement positions. It also does not verify facts independently. The uncased English checkpoint may normalize capitalization and handle specialized or multilingual terms poorly. These constraints define the demonstrated task; broader infilling methods are discussed later as separate approaches.

2.3 Functional and practical requirements

Table 2. Requirements of the implemented fill-mask assistant.

Requirement	Specification
Input	An English sentence with one [MASK] marker
Parameter	Integer top-k between 1 and 10
Validation	Non-empty text, valid mask count and encoded length
Prediction	Ranked vocabulary tokens for the masked position
Output	Token IDs, readable tokens, scores and completed sequences
Presentation	Browser interface, command line and JSON export

An invalid sentence should produce an explanatory error, not an invented replacement. The first run may download model files and take longer than later predictions. Network, memory and runtime failures must be recorded as execution problems. They are not evidence that the sentence has no suitable completion.

3. Design

3.1 System architecture

The design separates the interface from masked-token inference. Both app.py and cli.py call the MaskPredictor backend. The input layer collects the sentence and top-k value. Validation checks the request, the tokenizer prepares its numeric representation and the pretrained model scores the vocabulary at the masked position. The output layer returns the requested candidates and their decoded sequences.

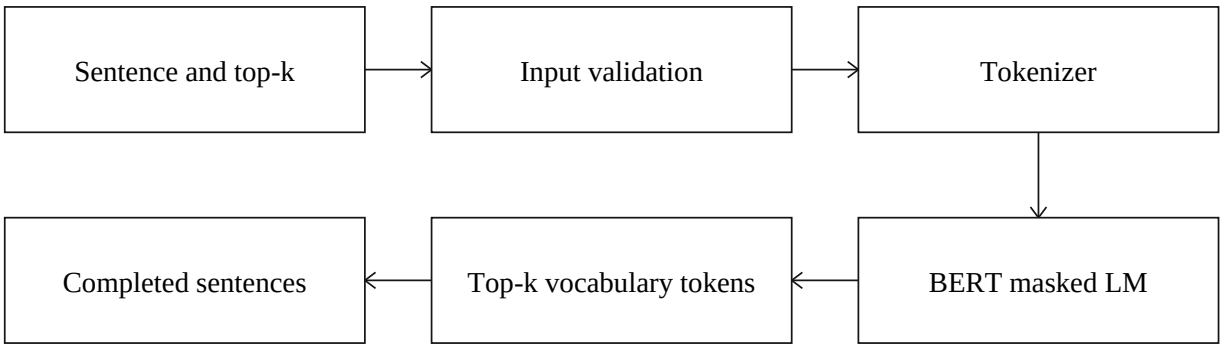


Figure 1. Processing architecture for single-mask prediction.

The object loads the pipeline only when needed and retains it for later requests. Streamlit caches this object across page reruns. The backend therefore remains usable from the terminal without depending on the browser interface, and interface behaviour can be tested with a controlled replacement for the model.

3.2 Prediction and presentation

Each candidate record contains token, token_str, score and sequence. The first is a numeric vocabulary identifier, the second is its readable representation, the third is the model's probability for that token and the fourth is a decoded completion. These fields allow the user to inspect both the model unit and its effect on the sentence.

The response also retains the input, top-k, model identifier, checkpoint revision and inference duration. Model loading and validation occur before the prediction timer starts. This duration is therefore not a measurement of total first-run latency. The record provides enough detail to distinguish model candidates from interface formatting.

3.3 Design rationale

The one-mask restriction keeps the target position unambiguous. The text-level check ensures that the user supplies the intended marker; an encoded check ensures that the tokenizer produces exactly one mask ID. These checks operate at different representations and protect the application contract. A literal marker can be counted correctly even when an incompatible tokenizer would not recognize it as intended.

The application preserves context rather than removing stop words or applying stemming. Such preprocessing could destroy grammatical cues. It also rejects overlength input instead of silently dropping words, since removed suffix context may substantially change a prediction. The uncased tokenizer's normalization is part of the selected checkpoint and should be explained when interpreting output.

3.4 From input tokens to vocabulary scores

The tokenizer creates token IDs and special markers. The model maps the sequence to contextual vector representations. At the masked position, the masked-language-model head converts its representation into logits for vocabulary entries. The task pipeline converts these logits into scores, selects the highest-ranked entries and decodes their candidate sequences [3].

This description separates two ideas: an encoder representation describes the input in context, while the prediction head maps that representation to candidate vocabulary entries. A contextual encoder can also support other tasks with suitable heads, but the existence of such tasks does not make this application a classifier or a question-answering system.

3.5 Comparing completion designs

Table 3. Completion designs discussed in the field; only the first is implemented.

Method	Target	Important distinction
Single-mask BERT	One vocabulary token	Directly matches this implementation
Several independent masks	Separate masked positions	Scores need not define a coherent joint phrase
Span reconstruction	A variable-length missing segment	Requires a method trained and decoded for spans
Iterative masked generation	A sequence reconstructed through repeated steps	Uses a separate generation process and objective

Adding more [MASK] markers is not a complete solution to phrase generation. If each position is predicted without conditioning on chosen replacements at the others, the resulting words may not form a coherent phrase. Iterative replacement changes the context between steps and creates an order-dependent procedure. A broader system must define and evaluate how it models those dependencies.

4. Proposed Methodology

4.1 Input preparation

The user enters a short sentence with [MASK] at the intended position and selects the number of suggestions. Leading and trailing whitespace is removed. The program rejects blank text, missing or repeated masks, and suggestion counts outside the supported range. It then tokenizes the sentence and verifies the encoded mask count and total length.

The sequence limit is measured in tokens, including special tokens, rather than words or characters. For the chosen BERT tokenizer, the usual configured maximum is 512 tokens [2]. The implementation reads tokenizer.model_max_length instead of assuming that every possible replacement checkpoint has the same limit. No automatic segmentation is applied, because splitting a sentence can remove the context needed for the gap.

4.2 Masked-position inference

Let z_v be the logit for vocabulary token v at the masked position and V the vocabulary. A softmax produces the conditional probability distribution over the vocabulary:

$$p(v \mid context) = exp(z_v) / \Sigma_{u \in V} exp(z_u) \quad (1)$$

The pipeline ranks vocabulary tokens by this distribution and returns the k highest values. The full distribution sums to one, but the displayed subset usually does not. If five scores sum to 0.60, the unshown entries collectively account for the remaining 0.40. That is not an output error; it reflects displaying a small subset of possible tokens.

As a mathematical illustration with only three hypothetical vocabulary entries, logits 2, 1 and 0 yield probabilities approximately 0.665, 0.245 and 0.090. Showing only two would display a total of approximately 0.910. These numbers explain the formula and are not the real BERT output. The application does not renormalize the displayed subset, because doing so would change the meaning of the scores.

4.3 Algorithm

Step 1: Read the masked sentence and top-k value.

Step 2: Check non-empty text, exactly one [MASK] and the supported integer range.

Step 3: Load or reuse the fill-mask pipeline.

Step 4: Tokenize and check one encoded mask and a valid sequence length.

Step 5: Run masked-position inference and obtain ranked candidate records.

Step 6: Preserve vocabulary scores and return tokens, sequences and metadata.

4.4 Evaluation procedure

A larger experiment can use 30 independently selected sentences with one hidden target token each. Include nouns, verbs, grammatical cues, ambiguous contexts and factual statements. Every removed target must correspond to exactly one tokenizer token; otherwise one mask would represent a different recovery problem. Sentences, gaps and targets should be fixed before comparing models or settings.

Top-1 exact recovery counts a match when the original target is the first candidate. Top-k recovery counts a match when it appears anywhere among k candidates. These measures assess recovery of the original text, not whether every other completion is unacceptable.

$$\textit{Top-k accuracy} = \textit{targets found in top-k} / \textit{total sentences} \tag{2}$$

Human review should supplement exact recovery. A reviewer can assess grammatical fit and contextual relevance separately. Where a sentence asserts a factual claim, check that claim using an independent source rather than model confidence. For example, several drinks can fit an ambiguous glass sentence, but the names of different cities are not equally correct answers to a specific capital question.

4.5 Measuring ambiguity and error categories

Errors can be grouped into failure to recover the original token, grammatical incompatibility, contextual mismatch, tokenization mismatch and factual inaccuracy. These groups are not identical. A grammatical alternative may be an exact-match failure, while a fluent factual error may receive a high score. Reporting only top-1 accuracy hides these differences.

Compare model outputs under controlled changes such as adding disambiguating context or moving the gap to a different part of the sentence. Changing top-k affects the number of displayed candidates, not the underlying learned weights. Increasing k makes exact recovery easier to count, so comparisons must use the same k and target-selection rule.

4.6 Reproducibility and limits of the experiment

Record the checkpoint revision, tokenizer, package versions, target token IDs and full candidate records. Target IDs avoid confusion from decoded spaces or normalized capitalization. Measure first-load time separately from steady-state inference and specify execution hardware when making speed claims. The current report makes no comparative latency claim because hardware-controlled benchmarking was not performed.

The evaluation collection should remain separate from examples used to select a model or adjust the experiment. An unfamiliar domain or vocabulary can change performance. Because the checkpoint was pretrained on large text collections, historical overlap with some common factual sentences may be unknown. A simple gap example should therefore be used to illustrate behaviour rather than to prove novel reasoning ability.

5. Implementation

5.1 Software environment

The project uses Python, PyTorch, Transformers, Streamlit and Pandas. The recorded local environment used Transformers 4.57.6, Streamlit 1.65.0, Pandas 3.0.6 and CPU-only PyTorch 2.14.1. These versions describe the implemented project and are not asserted to be the newest available. The README contains virtual-environment and installation instructions.

The selected pipeline task is fill-mask and the checkpoint is google-bert/bert-base-uncased. Model weights are downloaded when needed and are excluded from the GitHub repository. Cached files can support later local inference. The application itself contains no pretraining or fine-tuning procedure.

5.2 Core implementation and code explanation

Listing 1 uses the actual MaskPredictor backend from the repository folder. It shows the main prediction operation without duplicating the backend's validation logic.

Listing 1. Minimal use of the implemented MaskPredictor backend.

```
from predictor import MaskPredictor

model = MaskPredictor(device=-1)
result = model.predict("The capital of France is [MASK].", 5)

for item in result["predictions"]:
    print(item["token_str"], round(item["score"], 4))
    print(item["sequence"])
```

MaskPredictor stores the device, optional revision and a pipeline initially set to None. Its load method creates the Transformers pipeline once. The predict method checks the request, loads the model if required, obtains input_ids from the tokenizer and verifies the mask count and length. It then calls the pipeline with the sentence and top_k parameter.

The validation uses type(top_k) is int, which also rejects a Boolean supplied as a suggestion count. It checks the literal [MASK] marker before tokenization and the tokenizer's mask_token_id afterwards. Candidate IDs and scores are converted to ordinary Python numeric types before the response is serialized. Decimal rounding in the printed example affects readability only.

5.3 Repository structure and execution

Table 4. Main files in the public implementation.

File	Responsibility
predictor.py	Validation, loading and masked-token inference
app.py	Sentence input, top-k slider and result table
cli.py	Terminal parameters, JSON output and error reporting
evaluate.py	Original-token recovery on the fixed examples
examples/	Masked sentences, expected tokens and real predictions
tests/	Backend and interface behaviour checks

From the activated project environment, streamlit run app.py starts the browser interface. The command python cli.py "The capital of France is [MASK]." performs a terminal prediction. The --top-k option changes the number of suggestions, --output saves JSON and --revision can fix a model commit. The default --device value -1 selects CPU execution.

5.4 Interface and operational considerations

The page supplies a sentence input and a slider from 1 to 10. A form waits for the Predict replacements button. A cached predictor avoids repeated model initialization, and session state retains the result for display and download. The table contains readable tokens, scores and completed sentences so that the reader can inspect alternatives in context.

The interface removes stale results after a failed request and displays an error. The command line reports failures separately and returns a non-zero exit code. Model loading is excluded from the recorded inference duration. Download failures and unavailable memory must remain distinct from valid but inaccurate candidate lists.

5.5 Verification and model evaluation

Seven local tests check validation, length and encoded-mask handling, the expected inference contract and interface behaviour. Controlled backend replacements keep these checks deterministic. They verify application behaviour but do not assess BERT's language competence. The repository's GitHub workflow runs dependency-free checks; interface checks require Streamlit.

The evaluation script separately loads the real model. It tokenizes each expected answer without special tokens and requires a single vocabulary ID. It compares that ID with the first candidate and the five-candidate list, then saves per-sentence predictions in examples/sample_results.json. This is a measured model demonstration rather than a simulated interface test.

6. Result

6.1 Qualitative case analysis

Real CPU inference was completed on October 12, 2026 using checkpoint revision 86b5e0934494bd15c9632b12f734a8a67f723594. For "The capital of France is [MASK].", the saved output contains the following candidates. These are measured predictions rather than a list of manually selected reference completions.

Table 5. Measured fill-mask output for the France example.

Rank	Candidate token	Recorded score, rounded
1	paris	0.4168
2	lille	0.0714
3	lyon	0.0634
4	marseille	0.0444
5	tours	0.0303

The first decoded completion is "the capital of france is paris.". The other city names are statistically related alternatives, but are not correct answers to the factual capital question. Their presence demonstrates why ranking and truth should be evaluated separately. The displayed five scores do not sum to one because other vocabulary tokens retain probability mass.

6.2 Quantitative demonstration results

Table 6. Original-token recovery in the small implementation demonstration.

Measure	Observed value
Demonstration sentences	12
Original token recovered at rank 1	6 of 12
Top-1 exact recovery	50.0%
Original token found among five candidates	10 of 12
Top-5 exact recovery	83.3%

These results describe twelve hand-authored examples and are not a general accuracy estimate. Top-5 recovery is higher because it accepts the original target at any of five ranks. The experiment did not measure human acceptability, compare checkpoints or establish performance on an independent large collection. The 30-sentence procedure in Section 4 is proposed further evaluation, not an already completed experiment.

6.3 Interpretation and limitations

Exact recovery can underestimate usefulness when several completions fit. If an original sentence used water, milk might also produce a grammatical drink sentence. Conversely, fluency can conceal factual errors, as the alternative city names illustrate. A useful report therefore distinguishes matching the original token, fitting the sentence and stating a true fact.

Tokenization limits the task to a vocabulary entry at one position. Subword suggestions, capitalization normalization and domain-specific terms may complicate interpretation. The application does not replace a fragment with an assumed whole word or renormalize the visible scores. Predictions may also reflect biased associations in pretraining data. A larger evaluation should include such behaviours rather than only familiar factual examples.

6.4 Conclusion

The application demonstrates a reproducible single-mask workflow based on BERT: validate the input, encode the context, obtain vocabulary scores and inspect ranked completions. Its main educational value is the connection between contextual representations, the masked-language-model objective and actual prediction records. It is a baseline for studying completion behaviour, while broader sentence assistance requires evaluation of alternatives and methods that support multi-token gaps.

7. Current Status and Future Directions

7.1 Position of masked language modelling today

The literature and tools in this section were reviewed on October 13, 2026. BERT remains an accessible foundation for understanding bidirectional language representations, but the original checkpoint is not the endpoint of the field. Modern encoders and broader infilling methods continue to develop. The task of filling a single token, the objective used to train an encoder and a method for generating long text are related but distinct concepts.

RoBERTa revisited the BERT training recipe and demonstrated the importance of choices such as training data and optimization [4]. This established that changes in the training procedure can matter alongside architecture. A student should consequently identify the checkpoint and training objective rather than assuming that every encoder labelled BERT-like behaves identically.

7.2 Modern encoders and longer context

ModernBERT, published at ACL 2025, introduces updated encoder optimizations and a native context length of 8,192 tokens, with evaluations covering classification and retrieval [5]. It shows that encoder development continues beyond the original BERT recipe. Its reported downstream results do not imply that it will recover every masked token better in our sentence set; that would require running a matched completion experiment.

The longer context of a different encoder must not be attributed to our project. Our implementation uses bert-base-uncased and checks its tokenizer's configured limit. Replacing a checkpoint may also change the tokenizer, vocabulary and mask marker. A valid comparison would retest target-token eligibility, update input handling where necessary and hold the evaluation procedure consistent.

7.3 Masked diffusion and broader text generation

LLaDA, introduced in 2025, uses a forward masking process and a reverse token-prediction process to construct a diffusion language model [6]. It demonstrates a route from masked prediction to broader generation through a dedicated training and sampling framework. This differs from taking one BERT top-k list and treating it as a complete text generator.

The conceptual connection is that both use visible context to predict hidden information. The important difference is the sequence-level process: iterative generation must manage multiple positions and their changing context. This is an active research direction, not a capability added to the current single-mask application. Its quality and efficiency depend on model, sampling procedure and task-specific evaluation.

7.4 Recent infilling research

The 2026 MARIA study combines pretrained masked and autoregressive representations through a learned decoder to support infilling [7]. It illustrates research into using both bidirectional context and autoregressive infrastructure. The paper reports results on its own infilling tasks; these should not be presented as measured improvements to our BERT assistant.

Such work indicates that the field includes single-token recovery, span completion and methods that integrate different language-model objectives. Plain next-token prediction, masked-token prediction and a purpose-built infilling system should not be conflated. For a college project, the single-mask baseline offers a manageable entry point before attempting sequence generation and its more demanding evaluation.

7.5 Open challenges

Important unresolved practical issues include handling multi-token answers, specialized vocabulary, uncertainty, multilingual context and factual reliability. A model may allocate probability to several valid answers, and exact recovery alone cannot capture that ambiguity. Evaluation should consider token eligibility and acceptable alternatives rather than treating every difference from an original word as the same kind of failure.

Longer context and larger models also introduce resource trade-offs. A checkpoint that supports more tokens does not guarantee lower latency or better predictions on short sentences. Efficient execution, transparent output and reproducible evaluation remain important. Educational tools should make it easy to examine suggestions and correct unsuitable completions rather than implying that the first candidate is authoritative.

7.6 Future work and learning outcomes

A feasible next step is to run the independent 30-sentence experiment, include a human acceptability review and compare a second compatible masked-language-model checkpoint. Further work could investigate a domain-specific model or a span-infilling method, with clear target-selection and evaluation rules. Joint multi-token completion should be treated as a new design rather than a small change to the mask count.

After studying this topic, a student should be able to explain bidirectional context, distinguish training from inference, interpret top-k vocabulary scores and recognize why one token is not always one word. They should also distinguish the verified behaviour of the implementation from the broader capabilities discussed in research. These distinctions provide a foundation for understanding both encoders and newer infilling systems.

8. Reference

[1] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL-HLT, pp. 4171-4186, 2019. https://aclanthology.org/N19-1423/.

[2] Hugging Face, "google-bert/bert-base-uncased," model card. Accessed October 13, 2026. https://huggingface.co/google-bert/bert-base-uncased.

[3] Hugging Face, "Pipelines: FillMaskPipeline," Transformers 4.57.1 documentation. Accessed October 13, 2026. https://huggingface.co/docs/transformers/v4.57.1/en/main_classes/pipelines#transformers.FillMaskPipeline.

[4] Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv:1907.11692, 2019. https://arxiv.org/abs/1907.11692.

[5] B. Warner et al., "Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference," ACL, pp. 2526-2547, 2025. https://aclanthology.org/2025.acl-long.127/.

[6] S. Nie et al., "Large Language Diffusion Models," arXiv:2502.09992, 2025. https://arxiv.org/abs/2502.09992.

[7] D. M. Israel, A. Grover and G. Van den Broeck, "Enabling Autoregressive Models to Fill In Masked Tokens," Findings of EACL, pp. 4954-4965, 2026. https://aclanthology.org/2026.findings-eacl.260/.

Project implementation

Source code, setup instructions and recorded demonstration results:
https://github.com/Katori-bo/bert-fill-mask-assistant